Data Governance, Security & Compliance

A comprehensive framework for protecting data assets, enforcing access policies, and meeting regulatory obligations across modern data platforms.

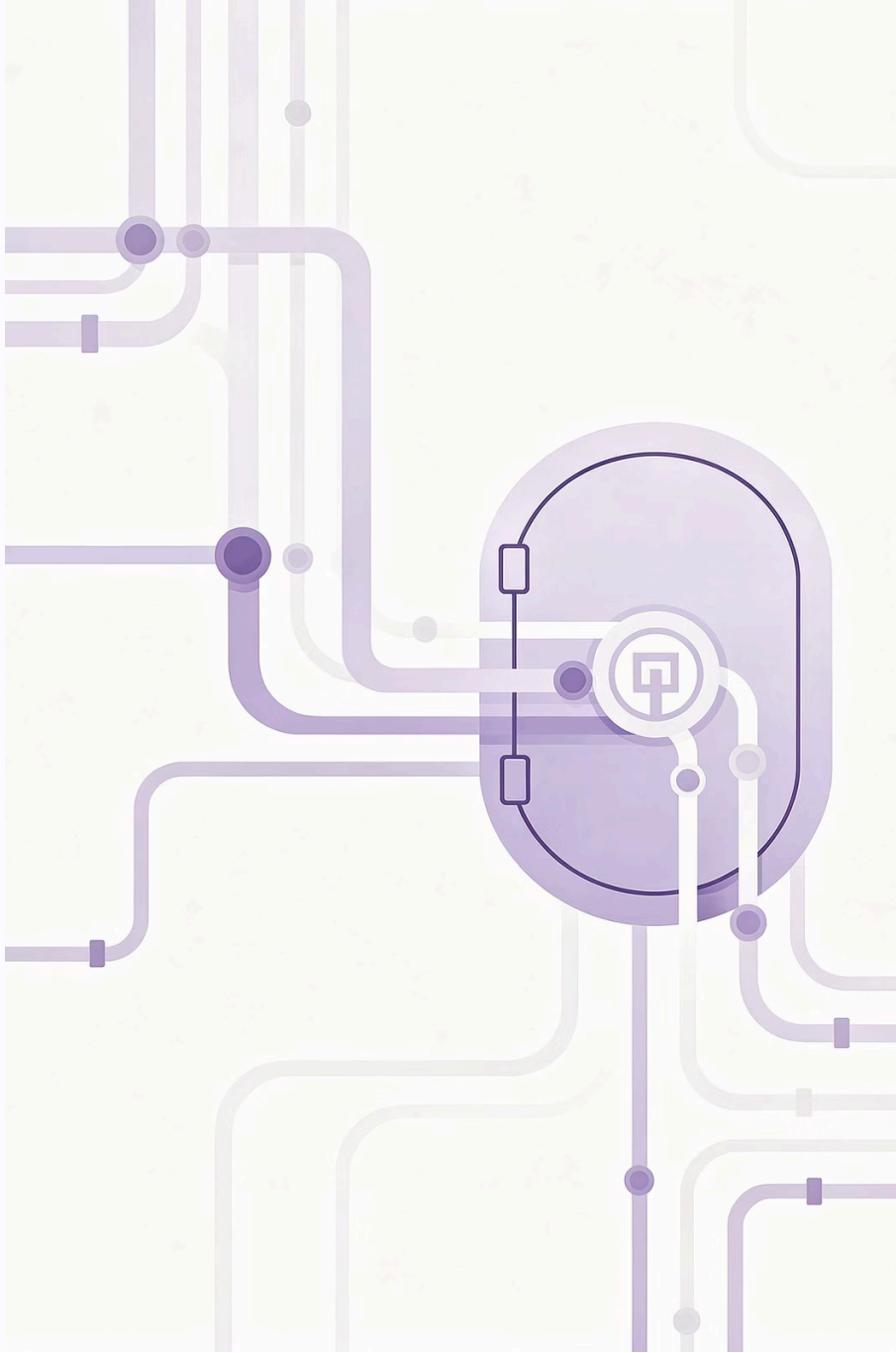
DATA ENGINEERING SERIES

Building on What We've Covered

This session brings together the threads from earlier in the series. We explored pipeline architecture and orchestration, then established the foundations of access control. Now we answer the critical question that ties it all together:

> **"How do we secure everything — end to end?"**

1. Pipeline design & orchestration
2. Access control fundamentals
3. End-to-end security & compliance



The Four Layers of Data Security

A robust security posture is never a single control. It is a defence-in-depth model in which each layer independently limits blast radius and collectively eliminates single points of failure. Every layer must be deliberately designed, not bolted on as an afterthought.

Network

VPC segmentation, private endpoints, firewall rules, and ingress/egress controls that limit who can reach your platform.

Storage

Encrypted object stores, managed disks, and bucket policies that protect data at its physical resting place.

Access

Identity-based controls, role hierarchies, and attribute-based policies that determine who can do what.

Data

Column masking, row filters, tokenisation, and classification labels that protect sensitive values directly within the dataset.

Access Control with Unity Catalog

Unity Catalog in Databricks provides a centralised, three-tier namespace — **catalog → schema → table** — and enforces access decisions at each level. Permissions are granted to users or service principals via role-based access control (RBAC), meaning that entitlements follow the identity rather than the object.

This model supports both coarse-grained and fine-grained controls within the same governance layer, dramatically reducing policy sprawl across workspaces.

RBAC — Unity Catalog

Grant `USE CATALOG`, `USE SCHEMA`, and `SELECT` privileges to roles, not individuals. Roles are then assigned to users or groups.

Table-Level Control

Restrict entire tables to specific teams — for example, only the Finance group may query the `transactions` table in production.

Column-Level Security

Apply dynamic column masks or deny `SELECT` on individual columns such as `ssn` or `credit_card_number` without duplicating the underlying table.

Access Hierarchy

Permissions cascade from catalog down to column. A deny at any level blocks access regardless of grants below it.

- Catalog admin
- Schema owner
- Table READ / WRITE
- Column mask function
- Row filter predicate

Data Security: Encryption at Rest & in Transit

Encryption is the last line of defence when other controls fail. It ensures that even if storage media is stolen or network traffic is intercepted, the underlying data remains unintelligible without the correct keys.

Encryption at Rest

All data written to cloud object storage (S3, ADLS, GCS) is encrypted using AES-256. Customer-managed keys (CMK) via AWS KMS, Azure Key Vault, or GCP Cloud KMS give organisations full control over the encryption lifecycle, including key rotation and revocation.

- Databricks-managed keys by default
- BYOK (Bring Your Own Key) for regulated workloads
- Delta Lake files, logs, and checkpoints all covered

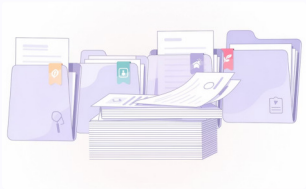
Encryption in Transit

All data moving between clients, clusters, and storage is protected by TLS 1.2 or higher. This covers Spark inter-node shuffle traffic, JDBC/ODBC connections to SQL warehouses, and REST API calls to the Databricks control plane.

- TLS enforced on all control and data plane traffic
- Certificate pinning available for high-security environments
- End-to-end encryption for notebook and cluster communications

Data Masking & PII Handling

Personally Identifiable Information (PII) requires deliberate handling throughout its entire lifecycle — from ingestion through transformation to serving. Masking strategies must align with both the sensitivity classification of the data and the legitimate business need of the consumer.



Classify & Tag

Use Unity Catalog tags or custom classification jobs to label columns as `PII`, `SENSITIVE`, or `CONFIDENTIAL`. Automated scanners (e.g., Databricks Lakehouse Monitoring) can detect patterns like emails, phone numbers, and SSNs.

Dynamic Masking

Apply column mask functions that return the real value to privileged roles and a redacted version — e.g., `XXX-XX-1234` — to all others. The mask is enforced at query time with zero data duplication.

Tokenisation & Pseudonymisation

Replace high-sensitivity identifiers with reversible tokens stored in a separate, tightly controlled vault. This satisfies GDPR pseudonymisation requirements while preserving analytical utility for authorised consumers.

Auditing: Who Accessed What, and When?

An audit trail is not optional — it is a regulatory and operational necessity. Databricks Unity Catalog emits a structured audit log for every data access event, capturing the principal, the object accessed, the operation performed, and the precise timestamp. These logs feed directly into your SIEM or data lake for long-term retention and anomaly detection.

01

Event Capture

Unity Catalog logs every `SELECT`, `INSERT`, `GRANT`, and schema change to a structured audit log stream in near real time.

02

Centralised Delivery

Audit events are delivered to cloud storage (S3/ADLS) or streamed to a SIEM such as Splunk, Microsoft Sentinel, or Datadog.

03

Query & Alert

Teams write SQL queries over audit tables to detect anomalies — e.g., a service account accessing PII columns outside business hours — and trigger automated alerts.

04

Retain & Report

Logs are retained per regulatory policy (typically 1–7 years) and used to produce evidence for compliance audits, penetration test reviews, and incident investigations.

Key Audit Fields

- **userIdentity** — who acted
- **actionName** — what they did
- **requestParams** — on which object
- **timestamp** — precise UTC time
- **sourceIPAddress** — origin of request
- **statusCode** — success or failure

Regulatory Compliance: GDPR & HIPAA

Non-compliance is not merely a legal risk — it is an existential one. Fines under GDPR can reach **€20 million or 4% of global annual turnover**, whichever is higher. HIPAA violations in the US can result in civil and criminal penalties reaching **$1.9 million per violation category per year**. Understanding what each regulation demands of your data platform is a prerequisite for every architecture decision.

GDPR — General Data Protection Regulation

- Applies to any organisation processing EU residents' personal data
- Requires purpose limitation, data minimisation, and storage limitation
- Right to erasure ("right to be forgotten") demands technical deletion workflows
- Data breach notification within 72 hours of discovery
- Privacy-by-design must be embedded in pipeline architecture

HIPAA — Health Insurance Portability & Accountability Act

- Governs Protected Health Information (PHI) in the US healthcare sector
- Security Rule mandates administrative, physical, and technical safeguards
- All PHI must be encrypted at rest and in transit
- Access logs (audit controls) are a mandatory safeguard
- Business Associate Agreements (BAAs) required with cloud vendors

Security Best Practices

Governance frameworks are only as effective as the operational disciplines that enforce them day to day. The following three principles form the backbone of a mature, sustainable security posture for any data platform.

Least Privilege

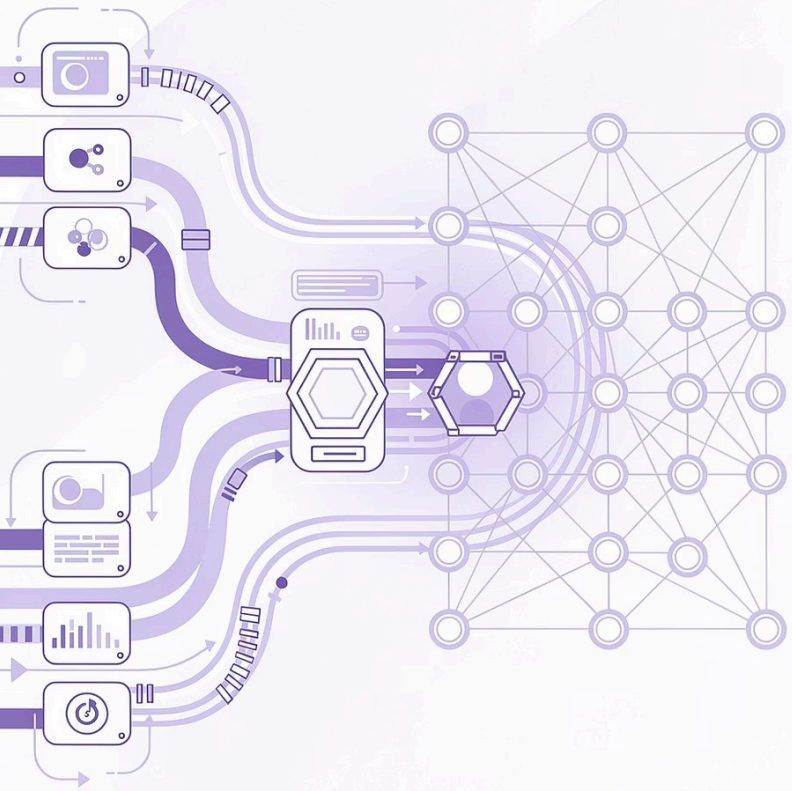
Grant every user, service principal, and automated job only the minimum permissions required to perform its function — nothing more. Regularly audit entitlements and revoke stale grants. Over-permissioned accounts are the most common root cause of data breaches in cloud environments.

Environment Isolation

Maintain strict separation between development, staging, and production environments using separate catalogs, workspaces, or cloud accounts. Data classified as production PII must never be accessible from a development environment without explicit, time-bound justification.

Continuous Monitoring

Implement automated anomaly detection over audit logs, cluster activity, and access patterns. Define alert thresholds for unusual query volumes, off-hours access to sensitive tables, and failed authorisation attempts. Treat security monitoring as a living system, not a one-time configuration.



What's Next: Data Sources & Real-Time Processing

With strong governance and secure foundations in place, the next step is to connect diverse data sources and design pipelines that process information as it arrives.

> **"Now let's explore how data sources flow into real-time processing pipelines — and how fresh, governed data powers fast decisions and responsive applications."**

Batch vs Streaming

Understanding when to use batch ingestion versus real-time streaming with tools like Kafka and Spark Structured Streaming.

Event-Driven Pipelines

Building pipelines that react to data events in real time using message queues and stream processors.

Real-Time Analytics

Delivering low-latency insights from live data streams directly to dashboards and downstream consumers.